SFTP Permissions Model

Permissions Model

Revision: 1 **Last modified:** 2026-07-11T15:23:48Z

The account permission system: the enum, the `public` guard, and every enforcement point from API to filesystem. Design-level until ATM-002/003 close with the captured container round-trip (RO upload denied / RW upload allowed).

Table of contents

1. Permission enum

Exactly two values — a closed set, validated at the API boundary (ATM-002) and in the config loader (ATM-003, unknown value = hard error):

Value	Capabilities inside the account's home
`read_only`	list, download
`read_write`	list, download, upload, rename, delete, mkdir

There is no `admin` SFTP-level permission: super-admin is an **API/console role only** and never an SFTP account capability. SFTP accounts are data-plane identities; the super-admin is a control-plane identity. They never mix.

2. The public flag

`public` marks an account for unauthenticated/anonymous-style access. Rules (operator mandate, ATM-002):

- Default is `false` in DB schema, API, web form, mobile form — **never** defaulted `true` anywhere.
- `true` requires `public_acknowledged: true` in the same write, or the console/mobile acknowledgement checkbox.
- `public: true` forces `permission: read_only`; the combination `public + read_write` is rejected with `422`.
- Every transition to `public: true` writes an audit row (actor, account, timestamp).
- A public account still lives under the same chroot + subdirectory layout (§4) — "public" relaxes authentication, never filesystem confinement.

3. Enforcement points

#	Point	What it enforces	Failure behavior
1	API request validation (ATM-002)	enum membership, public-guard, password policy	`400`/`422` with field-level message
2	DB constraint (ATM-002)	enum as check constraint; `public` default false	transaction abort
3	Config loader (ATM-003)	strict YAML/ JSON schema — unknown field or value = error	load fails loudly, no silent default
4	users.conf renderer (ATM-003)	one canonical line per account; hash-only passwords (`:e`)	atomic write; container never sees a partial file
5	Directory provisioner (ATM-003)	root-owned non-writable home top (chroot rule); writable subdirectory owned by	login refusal avoided (chroot modes correct); RO upload denied by filesystem

#	Point	What it enforces	Failure behavior
		account UID/GID; read_only mask on the writable subtree	
6	SFTP daemon	`ForceCommand internal-sftp` + chroot `%h`; no shell, no forwarding	session is SFTP-only, confined
7	Runtime round-trip (ATM-003/009 tests)	live container: RO `put` denied, RW `put` allowed	captured transcript per change; regression guard

A permission is therefore enforced **three independent ways** for writes: API admission (1–3), rendered config + filesystem masks (4–6), and the live-behavior proof (7). A regression in any one layer is caught by the others.

4. Directory + mask layout

Per account, under the host data root (`$SFTP_DATA_DIR`, default `./data`):

```
data/
└── <username>/            # chroot target: owned by root, mode
0755, NOT writable by the account
    └── upload/            # writable subtree: owned by
<uid>:<gid>
```

- `read_write`: `upload/` owned by the account UID/GID, mode `0755` — full write inside it.
- `read_only`: the writable subtree is rendered with a read-only mask for the account (write attempts fail at the filesystem even if a daemon bug admitted them) — the renderer + provisioner apply it on every permission change.
- Uploads at the chroot top level are impossible **by design** for every account (OpenSSH chroot rule) — this is a confinement guarantee, not a missing feature. Clients see this as “permission denied at `/`”; the documented working directory is `upload/`.

- UID/GID allocation is deterministic (managed by the renderer) so host and rootless-container views of ownership agree (rootless user-namespace mapping included).

5. Transition rules

Transition	Allowed	Side effects
create (any)	yes, super-admin only	DB row → render → provision → reload → audit
`read_write` → `read_only`	yes	re-render + mask tighten + reload + audit
`read_only` → `read_write`	yes	re-render + mask loosen + reload + audit
`public: false` → `true`	only with acknowledgement; forces `read_only`	re-render + audit (high-visibility row)
`public: true` → `false`	yes	re-render + audit
delete	yes; data retained unless `purge_data=true`	re-render + reload + audit
disable	yes	login blocked, data untouched, audit

In-flight sessions are never killed by a transition; new logins see the new state immediately after reload.

6. Evidence contract

The model is “true” only when all captured (§11.4 anti-bluff, §11.4.123):

1. Golden-file tests: renderer output for each enum × public combination (ATM-003).
2. Live container round-trip per permission change: RO `put` → denied; RW `put` → `100%` + byte-identical `get` (ATM-003 acceptance, `docs/qa/ATM-003/`).
3. Public-guard negative tests: `public+read_write` → `422`; `public:true` without acknowledgement → `422` (ATM-002 acceptance, `docs/qa/ATM-002/`).
4. Regression guards in the standing suite (ATM-009) re-run the round-trips on every release candidate.

7. Related docs

- overview.md — where enforcement points sit in the topology
- ../guides/user_management_guide.md — operator procedures
- ../guides/security_guide.md — chroot rule, threat model
- ../guides/troubleshooting_guide.md §3–§4 — permission failure playbooks

---

Sources verified (2026-07-11)

- Internal: `docs/Issues.md` ATM-002/003 (enum, public-guard, renderer, provisioner, acceptance criteria) · `docs/plans/master_implementation_plan.md` · `docs/research/mvp/MVP.md` (chroot/ownership baseline).
- External fetched this revision (via the guides): `https://github.com/atmoz/sftp` (users.conf grammar, `:e` marker, chroot home-not-writable symptom).